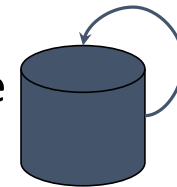


Duplicate Detection

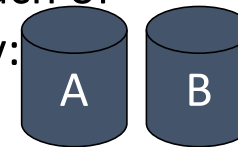
Khalid Belhajjame

CAUSES FOR DUPLICATES

1. Data about a single entity might be inadvertently entered multiple times into the same database: intra-source duplicates



2. Duplicates appear when integrating multiple data sources, each of which might have a different representation of a single entity: inter-source duplicates.



- Intra-source duplicates usually undergo some data fusion process to produce a new and enhanced single representation.
- Inter- source duplicates, on the other hand, are usually merely annotated as such.
 - Combining them is then left to the application that uses the data.

Intra-Source Duplicates

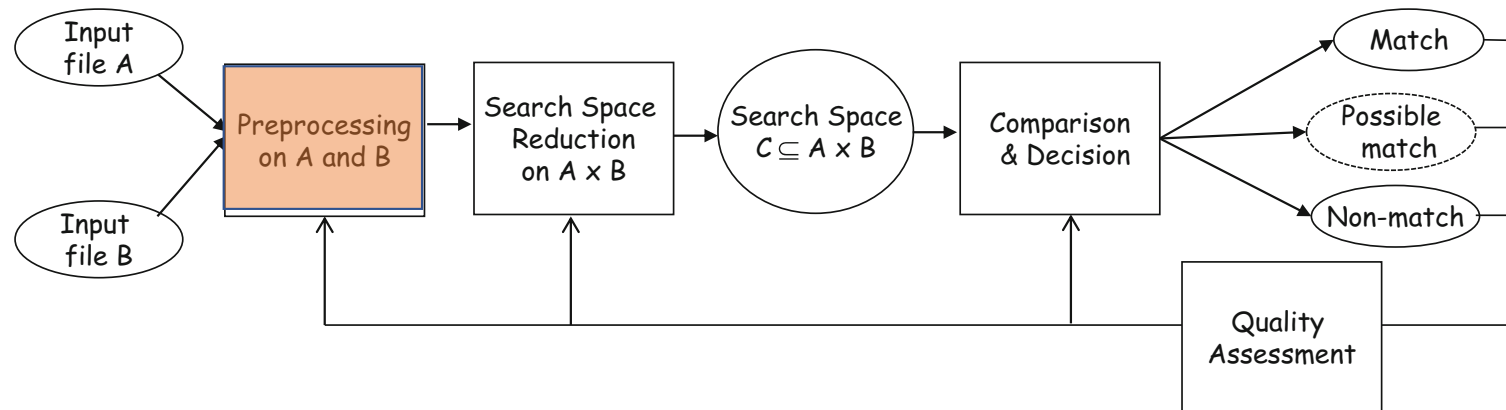
- Poor source entry: when entering customer data, for example:
 - inadvertent typos: 1969 vs. 1996
 - misheard words: Mohammed vs. Mohammad
 - difficult names: schwarzenegger.
 - lacking spelling abilities: Philadelphia
- Different conventions and data entry formats:
 - For example, when users, rather than trained data entry clerks, enter their data themselves, as is the case with many online shopping portals and other points of registration on the Web.

Inter-Source Duplicates

- Different data entry requirements
 - one source might allow abbreviated first names while the other enters the full first name.
 - numerical values might be recorded at different precision levels.
 - data might be entered in different languages.
- Different data entry times
 - Different sources might enter data about a real-world object at different points in time, e.g., address.
- Heterogeneous schemata
 - In general, different data sources use different schemata to represent same real-world entities.

High Level Process for entity deduplication

Consolidated techniques for deduplication have a common structure, described in the figure below, where we assume for simplicity we have two files as input information.



Preprocessing Step

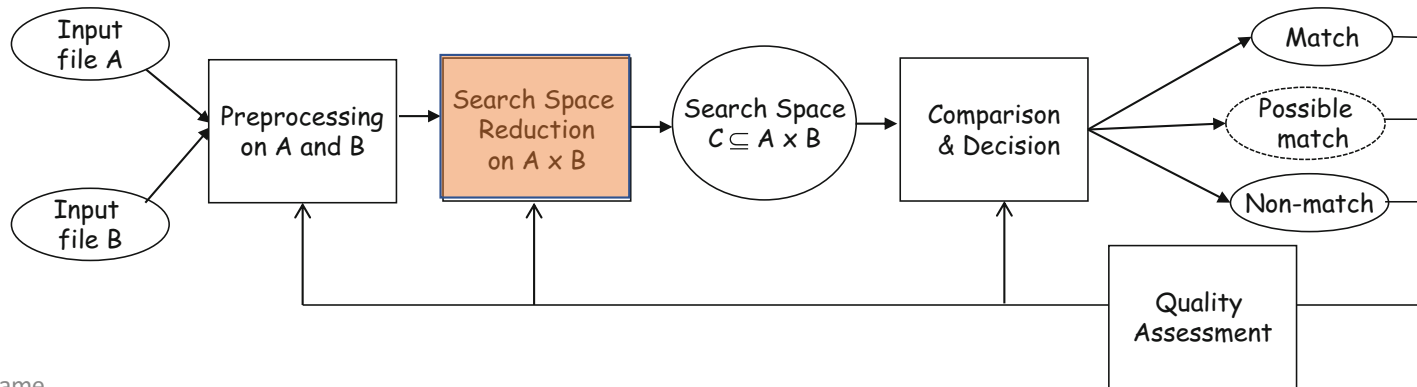
The preprocessing step includes the mainly standardization.

This consists of reorganization of composed fields, data type checks, and replacement of alternative spellings with a single one.

- A typical example of reorganization of a composite field is given by addresses. In many applications addresses are stored as a single string; the standardization activity may consider parsing the string into substrings corresponding, for instance, to StreetName, CivicNumber, City, and State. In the context of object identification, this type of reorganization has the purpose of making comparisons easier.

High Level Process for entity deduplication

1. A preprocessing activity that has the goal of working on data in order to standardize it and correct evident errors.
2. At this point we recognize that performing record linkage on the whole potential search space, consisting of the cartesian product of tuples in input files, results in a complexity $O(n^2)$, where n is the cardinality of each of the input relations. To make record linkage feasible, we have to produce a reduced search space.



Search Space reduction

- Consider that we have one file (or relation) R, with **n** tuples.
- The number of comparisons to be made in order to identify duplicates is
$$\frac{n \cdot (n-1)}{2}$$
- The figure on the right illustrates the comparison to be made when $n = 20$.
- What is the number of comparisons to be made in this case ?

[illegible]

Search Space reduction

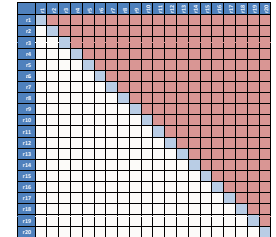
- Consider that we have two files (or relations) R and S, each with n tuples.
- The number of comparisons to be made in order to identify duplicates is $\frac{n(n-1)}{2}$.

$$\frac{n \cdot (n-1)}{2}$$

- The figure on the right illustrates the comparison to be made when $n = 20$.
- The number of comparisons to make in this case is 190

[illegible]

Search Space reduction

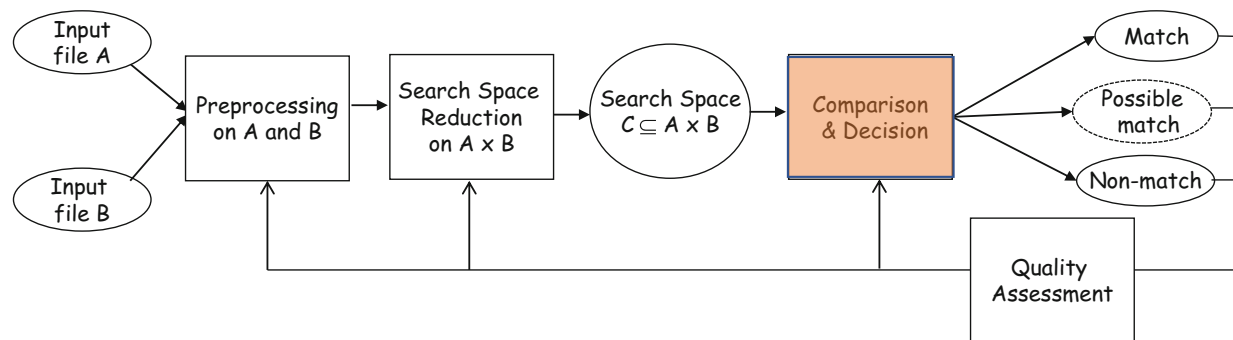


- Clearly, the quadratic time complexity as described is impractical for efficient duplicate detection on large databases.
- Consider, for example, a movie database with one million movie candidates.
- This would require approximately 5×10^{11} pairwise comparisons.
- Assuming a comparison can be done in 0.1 ms.
- The time for duplicate detection would roughly take one and a half years!
- Therefore, approaches to improve runtime have been devised. They all have in common that they aim at reducing the number of pairwise comparisons.
- We will discuss later some of these algorithms in more detail.

High Level Process for entity deduplication

3. Comparison and Decision

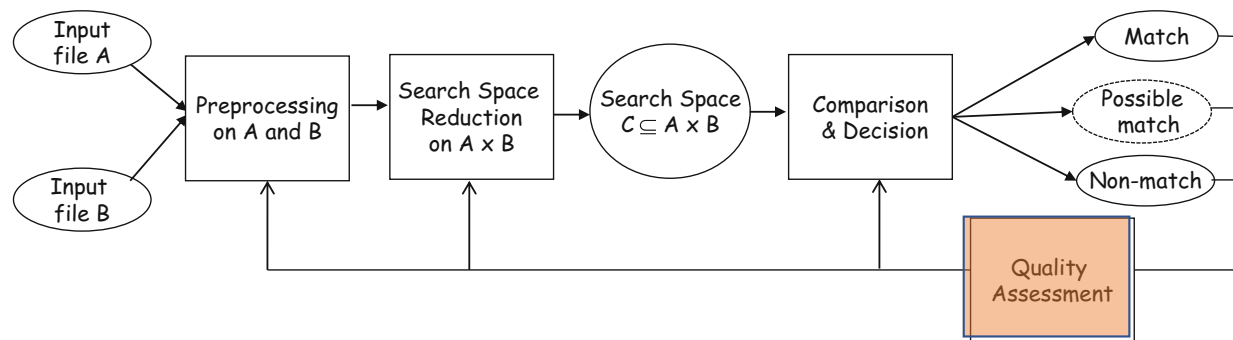
- The choice of a comparison function, or in the following comparison between records, that chooses the distance function and/or the set of rules that express the distance between pair of records in C .
- A decision is made among matching, unmatching, and possible matching based on the analysis of values of the comparison function.



High Level Process for entity deduplication

4. Finally, a quality assessment activity can be performed, during which measures are evaluated to assess if the result is satisfactory.

- Minimization of false positives, i.e., false assignments of pairs of tuples to the match decision, and the complementary false negatives.
- When the quality of the process is not satisfactory, it can be iterated, e.g., by adopting a new comparison function



Similarity Functions

- Given a similarity threshold θ , we classify two candidates c and c' using a similarity measure sim by:

$$\text{classify}(c, c') = \begin{cases} c \text{ and } c' \text{ are duplicates} & \text{if } \text{sim}(c, c') > \theta \\ c \text{ and } c' \text{ are non-duplicates} & \text{otherwise} \end{cases}$$

- In practice, more than two classes may be used, for instance, a third class for “possible duplicates” is often used.
- A distance measure, $\text{dist}(\cdot)$ measures exactly the opposite of similarity: The larger the distance between two candidates, the less likely it is that they are duplicates.

$$\text{classify}(c, c') = \begin{cases} c \text{ and } c' \text{ are duplicates} & \text{if } \text{dist}(c, c') \leq \theta \\ c \text{ and } c' \text{ are non-duplicates} & \text{otherwise} \end{cases}$$

- In the case of normalized similarity measures that return a similarity between zero and one, a corresponding distance score can be computed as $\text{dist}(c, c') = 1 - \text{sim}(c, c')$

String-Based Distance Functions

- **Edit Distance.** The edit distance between two strings is the minimum cost of converting one of them to the other by a sequence of character insertions, deletions, and replacements. Each of these modifications is assigned a cost value.
 - As an example, assuming that the insertion cost and the deletion cost are each equal to 1, the edit distance between the two strings Smith and Sitch is 2, as Smith is obtained by adding m and deleting c from Sitch.

Levenshtein

- Distance between strings s and t = shortest sequence of edit commands that transform s to t
 - Copy character from s over to t
 - Delete a character in s (cost 1)
 - Insert a character in t (cost 1)
 - Substitute one character for another (cost 1)

[illegible]

String-Based Distance Functions

- **n-Grams.** The n-grams comparison function forms the set of all the substrings of length n for each string. The distance between two strings is defined as $\sqrt{\sum_x |f_{s'} - f_{s''}|}$, where $f_{s'}$ and $f_{s''}$ are the number of occurrences of the substrings x in the strings s' and s'' , respectively.
- **Soundex Code.** The purpose of the soundex code is to cluster together names that have similar sounds. For example, the pronunciations of Hilbert and Heilbpr are similar.
 - Exemple: “Kuweit”, “Q8”

Item-Based Distance Functions

- Edit distance and other measures that consider the data to be compared as alphabetic strings can lead to distance measures that do not match with our intuition and practical situations.
- Let us consider the following example.

...	...	...	...	...
AT&T	...	...	AT&T Corporation	...
IBM Corporation			IBM Corporation	
...	...	...		

- If we want to link the two tables using as distance function the edit distance and normalizing the distance with the length 16 of the longest string, in the comparison of string AT&T Corporation with the two strings of the first table, we have

IBM Corporation -> AIBM Corporation -> ATBM Corporation -> AT&M Corporation -> AT&T Corporation: $4/16 = 0.25$

Where are distance between AT&T and AT&T Corporation is equal to $12/16$

- That is, “IBM Corporation” is closer than “AT&T” to “AT&T Corporation,” leading to an unintuitive conclusion.

Item-Based Distance Functions

- We can introduce a second metric that considers values no longer as alphabetic strings, but as groups of items, separated by blank characters, being in such a way a token-based distance function.
- This different view of data leads to a new distance called Jaccard distance, proposed more than one century ago, that can be evaluated with a two-step procedure:
 1. Split strings s_i and s_j in list of items, we call, e.g., LIT_i and LIT_j the lists of items corresponding to s_i and s_j .
 2. The Jaccard distance between two values s_i and s_j is

$$\text{JaccardDistance}(s_i, s_j) = 1 - \frac{|LIT_i \cap LIT_j|}{|LIT_i \cup LIT_j|}$$

- In this case, we have

$\text{JaccardDistance}(\text{"AT\&T"}, \text{"AT\&T Corporation"}) = ?$

$\text{JaccardDistance}(\text{"IBM Corporation"}, \text{"AT\&T Corporation"}) = ?$

Item-Based Distance Functions

- We can introduce a second metric that considers values no longer as alphabetic strings, but as groups of items, separated by blank characters, being in such a way a token-based distance function.
- This different view of data leads to a new distance called Jaccard distance, proposed more than one century ago, that can be evaluated with a two-step procedure:
 1. Split strings s_i and s_j in list of items, we call, e.g., LIT_i and LIT_j the lists of items corresponding to s_i and s_j .
 2. The Jaccard distance between two values s_i and s_j is

$$\text{JaccardDistance}(s_i, s_j) = 1 - \frac{|LIT_i \cap LIT_j|}{|LIT_i \cup LIT_j|}$$

- In this case, we have

- There are missing spaces

$$\text{JaccardDistance}(\text{"AT\&T"}, \text{"AT\&TCorporation"}) = 1 - \frac{1}{2} = 0.5$$

$$\text{JaccardDistance}(\text{"IBMCorporation"}, \text{"AT\&TCorporation"}) = 1 - \frac{1}{3} = 0.66$$

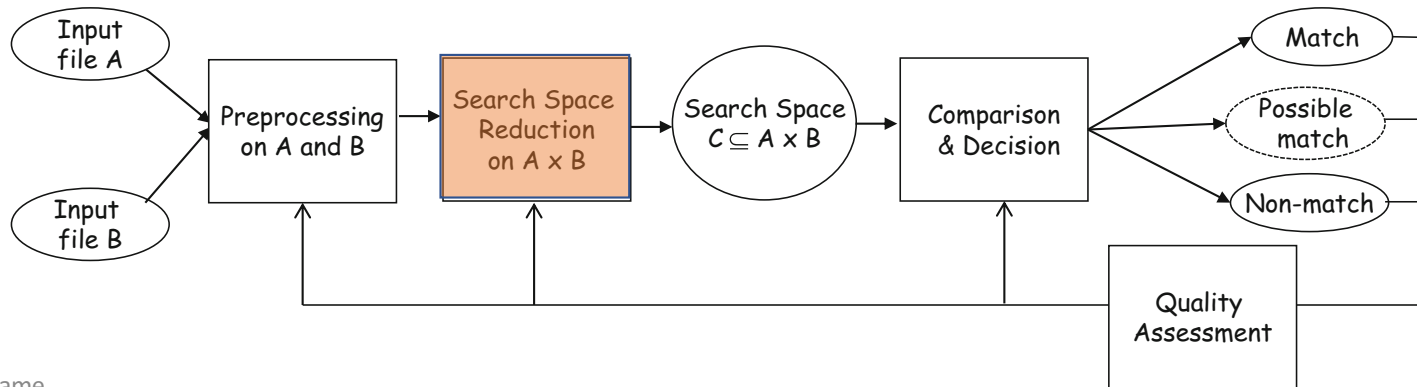
RULE-BASED RECORD COMPARISON

- Up to this point, we have discussed similarity measures and distance measures that compute a real-valued score. This score is then input to a duplicate classifier.
- We now discuss a complementary method to similarity measures for classifying candidate pairs as duplicates or non-duplicates.
- Rule-based record comparison approaches build rules on attributes (or combinations of attributes) to make a classification.
- An example of such a rule for two Person candidates c_1 and c_2 is on the right side.

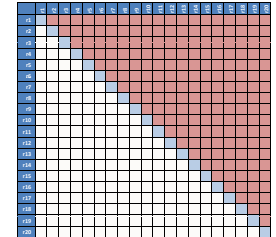
first name of c_1 is similar to first name of c_2
 $\wedge$ *last name of c_1 equals last name of c_2*
 $\Rightarrow$ *c_1 and c_2 are duplicates*

High Level Process for entity deduplication

1. A preprocessing activity that has the goal of working on data in order to standardize it and correct evident errors.
2. At this point we recognize that performing record linkage on the whole potential search space, consisting of the cartesian product of tuples in input files, results in a complexity $O(n^2)$, where n is the cardinality of each of the input relations. To make record linkage feasible, we have to produce a reduced search space.



PAIRWISE COMPARISON ALGORITHMS



- To avoid a prohibitively expensive comparison of all pairs of records, a common technique is to carefully partition the records into smaller subsets.
- Duplicate records are then examined only within the same partition.
- Generally, two solutions are used for this purpose:
 1. **Blocking methods** strictly partition records into disjoint subsets, for instance, using zip codes as partitioning key.
 2. **Windowing methods** sort the data according to some key, such as zip code, and then slide a window of fixed size across the sorted data and compare pairs only within the window.

Blocking methods

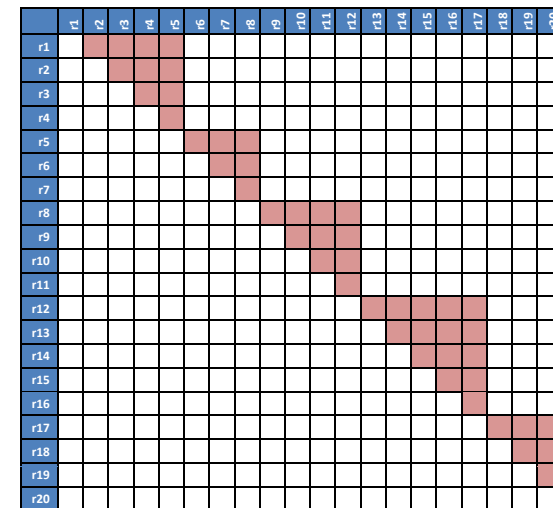
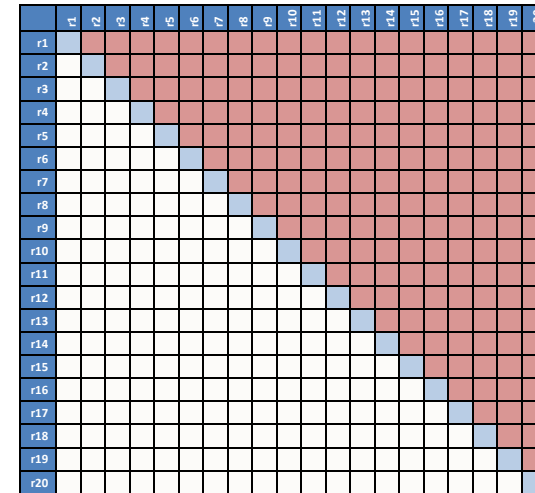
- Blocking methods pursue the simple idea of partitioning the set of records into disjoint partitions (blocks) and
- Then, comparing all pairs of records only within each block.
- Thus, the overall number of comparisons is greatly reduced.

Blocking methods

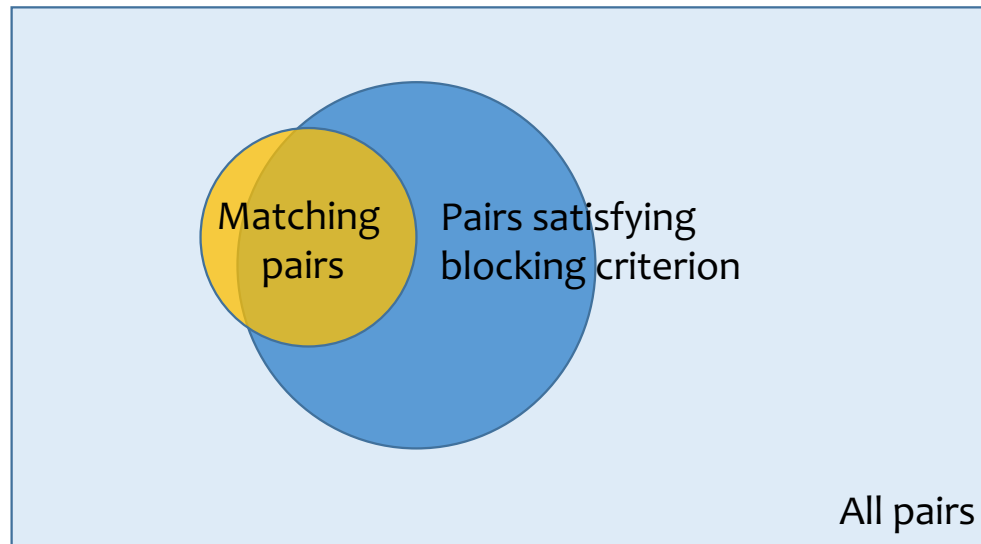
- Given n records and b partitions, the average size of each partition is n/b .
- This yields a total number of pairwise comparisons of:

$$b \cdot \frac{\frac{n}{b}(\frac{n}{b} - 1)}{2} = \frac{n(\frac{n}{b} - 1)}{2} = \frac{1}{2} \left(\frac{n^2}{b} - n \right)$$

- The figure on the right shows the number of comparisons to be made given 20 records that are partitioned into 5 blocks that contains 3 to 5 records.
- Notice that the number of pairwise comparisons dropped from 190 to 47



Blocking: The Conceptual Idea



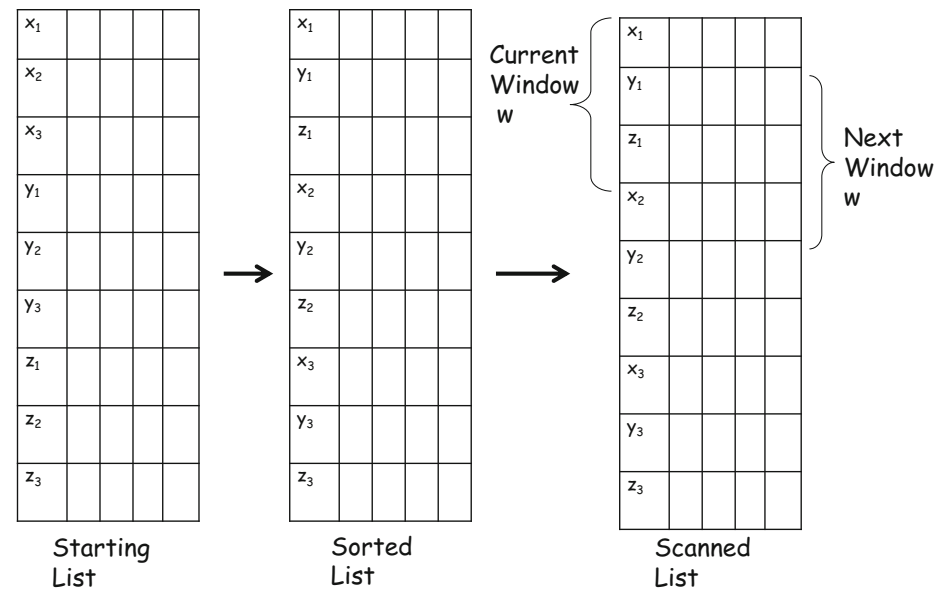
Try to reduce false positives (thereby reducing time) while avoiding false negatives

Blocking Methods

- An important decision for the blocking method is the choice of a good partitioning predicate, which determines the number and size of the partitions.
- They should be chosen in a manner that potential duplicates appear in the same partition.
- For example, for CRM applications a typical partitioning is by zip code or by the first few digits of zip codes.
 - The underlying assumption is that duplicates have the same zip code, i.e., there is no typo in the zip code and the customer has not moved from one zip code to another.
- Other partitionings might be by last name or some fixed-sized prefix of them, by street name, by employer, etc.
- To drop the assumption that duplicates have the same partitioning key, a multi-pass method can be employed. That is, the blocking algorithm is run multiple times, each time with a different partitioning key.
 - The chance that a duplicate pair does not appear together in at least one of the partitions is very low. It would have to contain errors in every attribute that is used for partitioning.

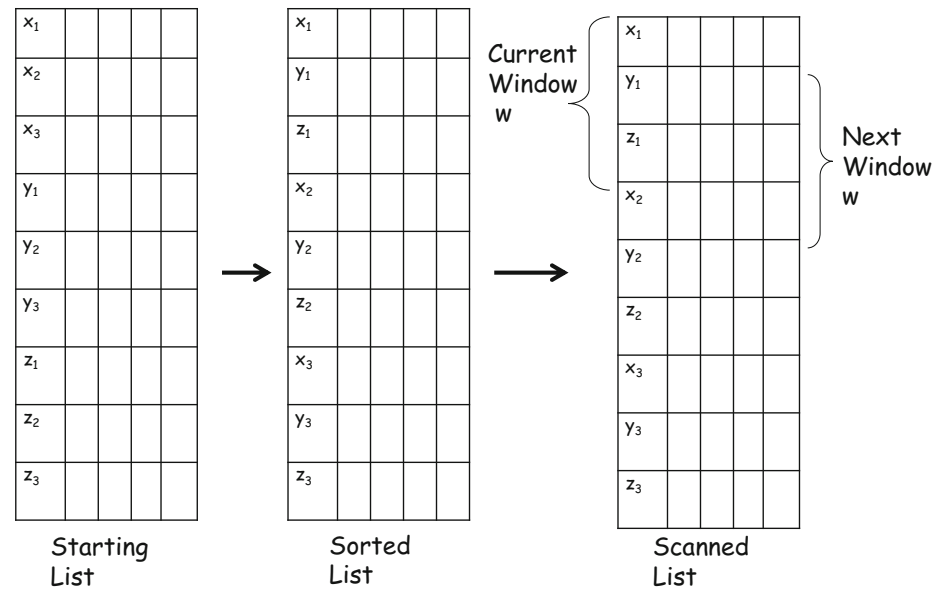
Sorted Neighborhood Method (SNM)

- **Sort data.** On the basis of a key that is specified, records are sorted in the data list.



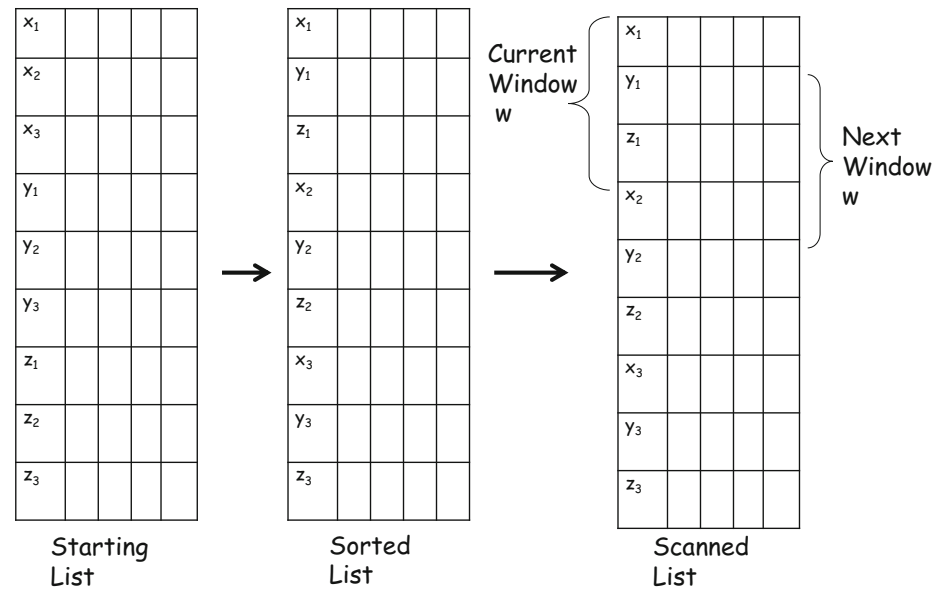
Sorted Neighborhood Method (SNM)

- **Merge.** A fixed size window is moved through the sequential list of records, limiting the comparisons for matching records to those records in the window (see the figure). If the size of the window is w records, then every new record entering the window is compared with the previous records to find matching records.



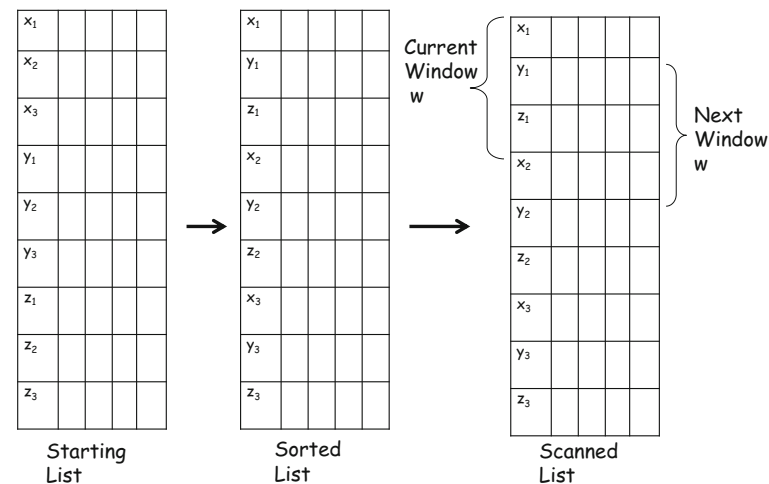
Sorted Neighborhood Method (SNM)

- **Merge.** If the size of the window is w records, then every new record entering the window is compared with the previous $w - 1$ records to find matching records.



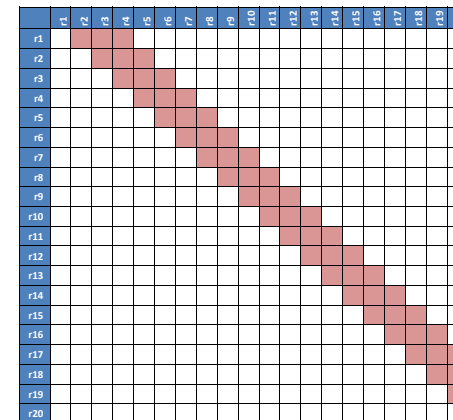
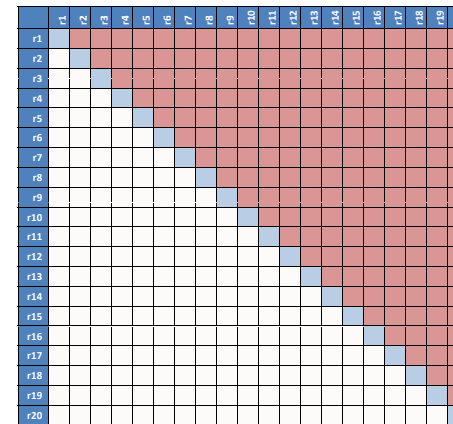
Sorted Neighborhood Method (SNM)

- In addition to the comparison performed in the merging phase, a transitive closure step is performed.
- Specifically, if records r_1 and r_2 are found to be similar and records r_2 and r_3 are also found to be similar, then r_1 and r_3 are marked to be similar as well.
- Note that while the couples (r_1, r_2) and (r_2, r_3) must be within the same window to be declared as similar, the inferred similarity between (r_1, r_3) does not require the two records to belong to the same window.
- This property can be exploited in order to have smaller sizes for the scanning window, with invariant accuracy of the result.



Sorted Neighborhood Method (SNM)

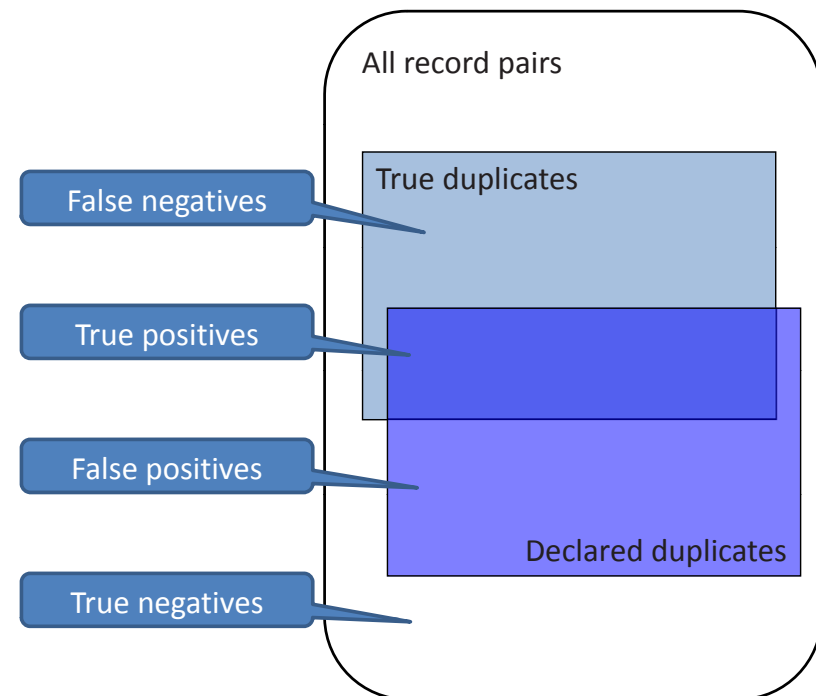
- The sorting key should be chosen distinct enough so that the number of records with the same key is not greater than the window size.
- Regarding the complexity, the Figure on the right illustrates the number of pairwise comparisons to be made given 20 records and a windows of size 4.
 - 54 pair-wise comparisons in total instead of 190



Qualities and Related Metrics

The decision on actual matching M or unmatching U of two records can give rise to two types of errors

- false positives FPs (also called false matches in the chapter) for records declared as M while actually being U and
- false negatives FNs (false unmatches) for records declared as U while actually being M.
- true positives TP (true matches) are the correctly identified M and
- true negatives TN (true unmatches) are the correctly identified U.



Qualities and Related Metrics

- Several metrics to evaluate effectiveness of object identification techniques. The most typical metrics are recall and precision.
- Recall measures how many true positives are identified in relation to the total number of actual matches. It is given by:

$$\text{recall} = \frac{TP}{M} = \frac{TP}{TP + FN}.$$

- Precision measures how many true matches are identified in relation to the total number of declared matches, including erroneous ones (i.e., FPs):

$$\text{precision} = \frac{TP}{TP + FP}.$$

- In order to combine recall and precision, F-measure has also been proposed. It corresponds to the harmonic mean of recall and precision. More specifically, F-measure is given by

$$F\text{-measure} = \frac{2RP}{P + R}.$$